SOLUÇÕES CLOUD — ENTREGA COMPLEMENTAR

Testes Unitários de Arquitetura

ArchUnitTS validando Vertical Slice + Clean Architecture no backend do Mini CRM

Repositório: github.com/PedroZCosta/Mini-CRM
Branch: `vertical-slice/clean-architeture`
Outubro de 2026

Integrantes do grupo

- Nicolas Tancon
- Leandro Canha
- Pedro Henrique Z. Costa
- Lincoln Neto

1. O que foi pedido

Esta entrega complementa a refatoração para Vertical Slice + Clean Architecture já feita no backend: além de organizar o código nesses padrões, o enunciado pede **testes unitários de arquitetura** que verifiquem, de forma automatizada, que a estrutura do código realmente respeita as regras de dependência — não o comportamento das rotas (isso os testes de domínio e de integração já cobrem), mas *quem tem permissão de importar quem*.

O enunciado deu uma biblioteca por linguagem, no estilo do ArchUnit original (Java):

Linguagem	Biblioteca indicada
Java	ArchUnit
C#	ArchUnit.NET
Node	ArchUnitTS
Python	PyTestArch

O backend do grupo é Node.js (Azure Functions), então a biblioteca usada foi o **ArchUnitTS**. O nome do pacote publicado no npm é `archunit` (o projeto se chama ArchUnitTS, mas o pacote é só `archunit` — detalhe que gerou confusão na primeira busca, registrado no `PROMPT.MD`). O projeto também fala em "bff e microservices" — este grupo tem um único backend (as Azure Functions), então os testes foram aplicados nele.

A biblioteca é feita para TypeScript, mas também funciona em projetos JavaScript puro — que é o caso daqui (ESM, sem TypeScript). Para isso ela precisa de um `tsconfig.json` com `allowJs: true`, usado só para a análise estática (o `api/tsconfig.json` criado nesta entrega) — nada no runtime da API muda, continua Node puro.

2. Como rodar os testes

Passo a passo para reproduzir qualquer um dos prints pedidos neste documento:

1. Clonar o repositório e entrar na branch da entrega:

   ```
   git clone https://github.com/PedroZCosta/Mini-CRM.git
   cd Mini-CRM
   git checkout vertical-slice/clean-architeture
   ```

2. Usar Node.js 22 ou mais novo (`node --version`) e instalar as dependências da API:

   ```
   npm install --prefix api
   ```

3. Rodar todos os 14 testes de arquitetura de uma vez:

   ```
   node --test --test-reporter=spec api/test/arquitetura.test.mjs
   ```

   Saída esperada: 14 linhas começando com `✔` (checkmark verde no terminal) e, no fim, `pass 14` / `fail 0`.

4. Para rodar só um subconjunto (usado nos prints de cada aluno), acrescenta `--test-name-pattern="<trecho do nome do teste>"` ao mesmo comando — os comandos exatos estão em cada seção abaixo.

Como tirar cada print: rode o comando indicado num terminal (PowerShell, bash ou o terminal integrado do VS Code), espere a execução terminar, e tire o print da janela do terminal mostrando o comando digitado e a saída completa (os `✔` e o resumo `tests/pass/fail` no final). Cole o print no espaço indicado logo abaixo do comando correspondente.

3. As 14 regras de arquitetura

Implementadas em `api/test/arquitetura.test.mjs` (ver no GitHub, commit `292aad9`). Resumo de todas, antes do detalhe por aluno:

#	Regra	O que verifica
1	Sem ciclos	Nenhum arquivo de `src/` depende, direta ou indiretamente, de algo que depende dele de volta
2	`domain` × `adapters`	Entities não dependem de HTTP, Mongo ou massa mock, em nenhuma das 7 slices
3	`domain` × `application`	Entities não conhecem Use Cases
4	`domain` × frameworks externos	Nenhum arquivo de `domain/` importa `@azure/functions` nem `mongodb`
5	`application` × `adapters`	Dependency Inversion: use case depende do port, nunca da implementação concreta
6–12	Isolamento das slices	Um teste por slice — nenhuma das 7 importa arquivo de dentro de outra
13	Nome dos ports	Todo arquivo em `application/ports/` termina em `Repository.js`
14	Nome dos use cases	Todo arquivo em `application/use-cases/` termina em `UseCase.js`

As seções 4 a 7 dividem essas 14 regras entre os 4 alunos, na mesma área de responsabilidade de cada um na entrega anterior, com o trecho de código de cada regra e o print pedido.

Nicolas Tancon

Slice `clientes` — regras 2 e 6

Valida que a entidade `Cliente` não depende de Mongo/HTTP, e que a slice `clientes` não importa nada de outra slice.

Evidência de código: arquitetura.test.mjs, linhas 36–49 e linhas 97–113

```
test('domain não depende de adapters em nenhuma slice', async () => {
  const regra = arquivos()
    .inFolder('src/features/*/domain/**')
    .shouldNot()
    .dependOnFiles()
    .inFolder('src/features/*/adapters/**');
  const violacoes = await regra.check();
  assert.equal(violacoes.length, 0, formatar(violacoes));
});
```

PRINT 1 — NICOLAS

Regra "`domain` não depende de `adapters`" rodando sozinha.

```
node --test --test-reporter=spec --test-name-pattern="domain não depende
de adapters" api/test/arquitetura.test.mjs
```

O terminal deve mostrar `✔ domain não depende de adapters em nenhuma slice` e `pass 1` / `fail 0`.

[colar aqui o print do terminal]

PRINT 2 — NICOLAS

Isolamento da slice `clientes`.

```
node --test --test-reporter=spec --test-name-pattern='slice "clientes"'
api/test/arquitetura.test.mjs
```

O terminal deve mostrar `✔ slice "clientes" não depende de nenhuma outra slice` e `pass 1` / `fail 0`.

[colar aqui o print do terminal]

Leandro Canha

Slices `oportunidades` e `tarefas` — regras 3 e 6

Valida que as entidades `Oportunidade` e `Tarefa` não dependem dos próprios use cases, e que as duas slices não importam nada uma da outra nem de mais ninguém.

Evidência de código: arquitetura.test.mjs, linhas 51–63

```
test('domain não depende de application em nenhuma slice', async () => {
  const regra = arquivos()
    .inFolder('src/features/*/domain/**')
    .shouldNot()
    .dependOnFiles()
    .inFolder('src/features/*/application/**');
  const violacoes = await regra.check();
  assert.equal(violacoes.length, 0, formatar(violacoes));
});
```

PRINT 1 — LEANDRO

Regra "`domain` não depende de `application`" rodando sozinha.

```
node --test --test-reporter=spec --test-name-pattern="domain não depende
de application" api/test/arquitetura.test.mjs
```

O terminal deve mostrar `✔ domain não depende de application em nenhuma slice` e `pass 1` / `fail 0`.

[colar aqui o print do terminal]

PRINT 2 — LEANDRO

Isolamento das slices `oportunidades` e `tarefas`.

```
node --test --test-reporter=spec --test-name-pattern='slice
"oportunidades"|slice "tarefas"' api/test/arquitetura.test.mjs
```

O terminal deve mostrar as duas linhas `✔ slice "oportunidades" não depende...` e `✔ slice "tarefas" não depende...`, com `pass 2` / `fail 0`.

[colar aqui o print do terminal]

Pedro Henrique Z. Costa

Slices `dashboard` e `interacoes` — regras 5 e 6

Valida a Dependency Inversion (os use cases do dashboard dependem dos próprios ports, nunca de um adapter concreto de outra slice) e o isolamento das duas slices.

Evidência de código: arquitetura.test.mjs, linhas 82–95

```
test('application não depende de adapters em nenhuma slice', async () => {
  const regra = arquivos()
    .inFolder('src/features/*/application/**')
    .shouldNot()
    .dependOnFiles()
    .inFolder('src/features/*/adapters/**');
  const violacoes = await regra.check();
  assert.equal(violacoes.length, 0, formatar(violacoes));
});
```

PRINT 1 — PEDRO

Regra "`application` não depende de `adapters`" (DIP) rodando sozinha.

```
node --test --test-reporter=spec --test-name-pattern="application não
depende de adapters" api/test/arquitetura.test.mjs
```

O terminal deve mostrar `✔ application não depende de adapters em nenhuma slice` e `pass 1` / `fail 0`.

[colar aqui o print do terminal]

PRINT 2 — PEDRO

Isolamento das slices `dashboard` e `interacoes`.

```
node --test --test-reporter=spec --test-name-pattern='slice
"dashboard"|slice "interacoes"' api/test/arquitetura.test.mjs
```

O terminal deve mostrar as duas linhas `✔ slice "dashboard"...` e `✔ slice "interacoes"...`, com `pass 2` / `fail 0`.

[colar aqui o print do terminal]

Lincoln Neto

Slices `usuarios`, `health`, kernel e configuração do ArchUnitTS — regras 1, 4, 6, 13, 14

Responsável por instalar o `archunit`, criar o `api/tsconfig.json` e validar as regras de abrangência geral (sem ciclos, isolamento de frameworks externos, convenções de nome) — a mesma área de infraestrutura que ele organizou na entrega anterior (`shared/`).

Evidência de código: linhas 25–34, 65–80, 115–140, e a configuração em api/tsconfig.json

```
test('domain não importa @azure/functions nem mongodb diretamente', async
() => {
  const regra = arquivos()
    .inFolder('src/features/*/domain/**')
    .should()
    .adhereTo(
      (arquivo) => !/from\s+['"](@azure\/functions|mongodb)
['"]/.test(arquivo.content),
      'Entity não pode importar @azure/functions nem mongodb...',
    );
  const violacoes = await regra.check();
  assert.equal(violacoes.length, 0, formatar(violacoes));
});
```

PRINT 1 — LINCOLN (EXECUÇÃO COMPLETA)

Todos os 14 testes de arquitetura rodando juntos — a evidência geral da entrega.

```
npm install --prefix api
node --test --test-reporter=spec api/test/arquitetura.test.mjs
```

O terminal deve mostrar as 14 linhas com `✔` e, no resumo final, `tests 14` / `pass 14` / `fail 0`.

[colar aqui o print do terminal]

PRINT 2 — LINCOLN

As regras de abrangência geral: sem ciclos, sem import de frameworks externos no `domain`, e as duas convenções de nome.

```
node --test --test-reporter=spec --test-name-pattern="ciclos de
dependência|domain não importa|ports em application|use cases em
application" api/test/arquitetura.test.mjs
```

O terminal deve mostrar 4 linhas com `✔` e `pass 4` / `fail 0`.

[colar aqui o print do terminal]

PRINT 3 — LINCOLN

Isolamento das slices `usuarios` e `health`.

```
node --test --test-reporter=spec --test-name-pattern='slice
"usuarios"|slice "health"' api/test/arquitetura.test.mjs
```

O terminal deve mostrar as duas linhas correspondentes e `pass 2` / `fail 0`.

[colar aqui o print do terminal]

4. Print extra (opcional): provando que o teste pega violação de verdade

Um teste de arquitetura que nunca falha pode só estar mal escrito, não necessariamente provando nada. Este print opcional mostra o teste **falhando** de propósito, como evidência extra de que ele realmente verifica a regra (foi assim que o grupo validou cada regra antes de confiar nela — ver `PROMPT.MD`, Prompt 7).

1. Abra `api/src/features/oportunidades/domain/Oportunidade.js` e acrescente, como a primeira linha do arquivo:

   ```
   import { MongoClienteRepository } from
   '../../clientes/adapters/persistence/MongoClienteRepository.js';
   ```

2. Rode:

   ```
   node --test --test-reporter=spec api/test/arquitetura.test.mjs
   ```

   Duas linhas devem aparecer com `✖` (falha): a regra de `domain` / `adapters` e o isolamento da slice `oportunidades` — as outras 12 continuam `✔`.

3. **Desfaça a alteração** (remova a linha adicionada) antes de continuar usando o projeto — este passo é só para o print, não é uma mudança real no código.

PRINT EXTRA (OPCIONAL)

Execução mostrando exatamente 2 testes falhando (`not ok` / `✖`) e os outros 12 passando, provando que a suíte reage a uma violação de verdade.

[colar aqui o print do terminal, se for incluir esta evidência extra]